Enterprise Doc Q&A Assistant — Engineering Blueprint

Oct 31, 2026 · @Rishikesh

Recommendation in one line: a single-process Python app — Streamlit UI calling a FastAPI backend that wraps an in-process RAG engine (section-aware chunking, hybrid BM25 + local embeddings, evidence gate, one JSON-mode LLM call, citation validation, rule-based conflict hints) — built by three people in parallel against one frozen schema file.

0. Assumptions and how to read this

- **Time budget.** Your message says about 5 hours to build, but Stage 8 asks for an 8-hour plan. The schedule is built for **5 hours** (the binding constraint), with clearly marked extensions if you actually get 8.
- **Knowledge collection.** The brief says "the supplied knowledge collection". If organisers hand you documents, use them. If not, you build a small synthetic corpus — the brief explicitly allows synthetic questions and labels, as long as you document how you made them.
- **Hardware.** CPU-only laptops, Python 3.11, internet access for one hosted LLM API. No GPU assumed.
- **API access.** You need at least one LLM API key with free-tier or paid quota. A consumer chat subscription (Claude Pro, Gemini Advanced, ChatGPT Plus) does **not** include API access. Details in Stage 4.
- **Labels used throughout: [M]** = mandatory, traced to a numbered requirement (FR-01…FR-08) or a deliverable in the brief. **[R]** = recommended by me, not in the brief. **[O]** = optional, only if time remains.

Stage 1 — Understanding the problem

The judges are not asking for a chatbot; they are asking for a **trustworthy** answerer that proves every claim, shows disagreement between documents, and says "I don't know" when it should — and then measures itself.

1.1 What the problem actually is

Employees ask about account access, configuration, billing, usage limits and troubleshooting. The answers are scattered across manuals, FAQs, release notes and support articles, and those sources are **not always consistent or current**. The real pain is not search — it is *finding the correct and current guidance* and trusting it.

So the system has three jobs: find the right passages, answer only from them, and be honest about gaps and contradictions.

1.2 What you are expected to build

A web interface **or** an API that accepts documents and natural-language questions and returns a grounded answer with citations. A web UI is not strictly required by the brief, but for a live demo it is the obvious choice **[R]**.

1.3 Mandatory functional requirements (from the brief)

ID	Capability	What it means in practice	Hard part
FR-01	Multi-format ingestion	Ingest several PDF, HTML, Markdown, TXT files; keep title, section and source metadata	Each format exposes structure differently
FR-02	Content preparation	Clean and split into passages without losing headings or context	Chunk boundaries that keep meaning
FR-03	Natural-language retrieval	Rank passages for a question, including paraphrases and terminology variations	Synonyms vs exact codes/terms
FR-04	Grounded answers	Concise answer from retrieved evidence only; no unsupported claims	LLM hallucination control
FR-05	Citations	Per claim: document, section or page where available, excerpt, link or identifier	Mapping each sentence to a source
FR-06	Conflict handling	Detect contradictory or version-sensitive passages; show the conflict instead of picking one	Hardest requirement

ID	Capability	What it means in practice	Hard part
FR-07	Confidence and fallback	Flag weak evidence, ask for missing context, or return a clear not-found	Calibrating thresholds
FR-08	Evaluation	Labelled question set incl. answerable, ambiguous, unanswerable; measure retrieval and answer quality	Building gold labels fast

Note the wording "**for each claim**" in FR-05 — a citation list at the bottom of the answer is weaker than claim-level citations. Design for claim-level.

1.4 Expected deliverables (all mandatory)

1. **Working prototype** — web UI or API: documents in, question in, grounded answer with citations out.
2. **Evaluation report** — test set, retrieval metrics or ranking analysis, answer-quality method, failure cases, limitations.
3. **Evidence-backed examples** — at least 5 questions covering: direct answer, paraphrase, conflicting evidence, and an unsupported question.
4. **Technical documentation** — README with setup, supported formats, indexing and retrieval approach, assumptions, design trade-offs.

Also required by the scope section: if you create synthetic questions or relevance labels, **document how and with what assumptions**.

1.5 What the evaluator will look for

The brief's five judging dimensions, and how a judge will probably test each live:

Dimension	Likely live test	What passes
Retrieval quality	Asks a paraphrased question	Correct passage in top 3, shown on screen
Answer groundedness	Clicks a citation	Excerpt really supports the sentence it is attached to

Dimension	Likely live test	What passes
Answer usefulness	Reads the answer	Short, direct, complete — not padded
Conflict handling	Asks about something two docs disagree on	Both positions shown with their versions/dates
Fallback behaviour	Asks something not in the docs, or something vague	Clear "not found" or a clarifying question — no invented answer

Notice the phrase "without rewarding unsupported fluency". A beautiful wrong answer scores **worse** than a short honest one.

1.6 Average vs excellent

Average solution	Excellent solution
Fixed-size chunks, headings lost	Section-aware chunks carrying "Doc > Section > Subsection" breadcrumb and page numbers
Vector search only	Hybrid (keyword + semantic) with a measured comparison showing why
One citation list under the answer	Citation on every sentence, each linking to doc, section, page and excerpt
Conflicts ignored or left to luck	Conflicts detected by rules + LLM, shown side-by-side with version/date
Always answers	Evidence gate refuses before calling the LLM; asks clarifying questions
"We tested it manually"	Labelled set, Recall@k / MRR, ablation table, failure analysis

1.7 The technically challenging parts (ranked)

1. **Conflict detection (FR-06)** — deciding that two passages talk about the *same thing* and *disagree*. Pure LLM is unreliable; pure rules miss cases.
2. **Confidence calibration (FR-07)** — picking score thresholds that separate answerable from unanswerable. Needs the eval set to tune.

3. **Claim-level citations (FR-05)** — forcing the LLM to tie every sentence to evidence IDs, then validating them.
4. **Structure-preserving extraction (FR-01/02)** — PDF headings are not tagged; HTML has noise; TXT has no structure.
5. **Evaluation labels (FR-08)** — gold labels must survive re-chunking, or your metrics break every time chunking changes.

1.8 Mandatory vs optional at a glance

- **[M] Mandatory:** FR-01 to FR-08 and the four deliverables above.
- **[R] Recommended (not in brief, high value):** hybrid retrieval, evidence gate before the LLM, claim-level JSON output, a retrieval ablation table, an LLM response cache.
- **[O] Optional:** cross-encoder reranking, LLM-as-judge groundedness scoring, query rewriting, a document viewer, filters by product/version, streaming answers.

Stage 2 — Solution options and the recommendation

Recommended: **Hybrid RAG with an evidence gate** — plain Python, no orchestration framework, no vector database server, one LLM call per question.

2.1 Four candidate architectures

Option	How it works	Verdict
A. Long-context stuffing	Paste all documents into a large-context LLM per question	**Reject.** No retrieval to measure (FR-03, FR-08 need ranking), burns free-tier quota, weak per-claim citations, breaks as corpus grows
B. Basic vector RAG	Fixed chunks → embeddings → top-k → LLM	**Not enough.** Misses exact terms (error codes, plan names, "SSO"), no conflict or confidence logic
C. Hybrid RAG + evidence gate	Section-aware chunks → BM25 + dense, fused → confidence gate → structured LLM answer → citation validation → conflict hints	**Recommend.** Meets every FR, each stage is testable alone, ~1,000 lines of Python

Option	How it works	Verdict
D. Agentic / graph RAG	Query planner, multi-hop tool calls, knowledge graph, LangGraph	**Reject.** Several LLM calls per question, slow, hard to debug in 5 hours, no requirement needs it

2.2 Component-by-component decisions

Component	Use it?	Why
RAG architecture	**Yes [M]**	FR-03/04 literally describe retrieve-then-generate
Embedding model (local)	**Yes [M]**	Handles paraphrase ("can't sign in" ↔ "authentication failure"). Local = no quota, no network at query time
BM25 keyword retrieval	**Yes [R]**	Handles exact tokens embeddings blur: `ERR-403`, `Enterprise plan`, `SAML`. Costs ~20 lines
Hybrid fusion (RRF)	**Yes [R]**	Combines both rankings without tuning score scales; makes FR-03 "terminology variations" defensible
Vector database server (Chroma, Qdrant)	**No**	A hackathon corpus is < 10,000 chunks; brute-force cosine over a NumPy matrix takes a few milliseconds. A DB adds install, persistence and API surface for zero gain
FAISS	**Optional**	Same result as NumPy at this scale; swap in only if the organisers ship a huge corpus
Cross-encoder reranker	**Optional [O]**	Typically lifts top-3 precision and gives a cleaner confidence score, but adds a ~90 MB model and latency. Add in phase 2 only if eval shows a gain
LLM answer generation	**Yes [M]**	One call, JSON output, claim → evidence IDs

Component	Use it?	Why
Rule-based conflict detection	**Yes [M]**	Deterministic, explainable, catches version/date and numeric disagreements
LLM-assisted conflict detection	**Yes, same call**	The generation prompt also asks for conflicts; rules provide hints, LLM confirms. No extra call
LangChain / LlamaIndex	**No**	They speed up a toy demo but hide exactly the parts you are judged on (gates, claim citations, metadata). Version churn costs debugging time

2.3 Why this is the simplest architecture that meets every requirement

- Every FR maps to **one** module you can point at: parsers (FR-01), chunker (FR-02), retriever (FR-03), generator (FR-04), citation validator (FR-05), conflicts (FR-06), confidence gate (FR-07), eval runner (FR-08).
- Only **one** network dependency (the LLM). Retrieval, ranking, confidence and conflict rules all run locally, so they work even if the API is rate-limited.
- Hallucination is fought in four layers, not one: (1) refuse before calling the LLM if evidence is weak, (2) prompt that permits only numbered evidence, (3) JSON output forcing a citation per claim, (4) post-validation that drops claims citing nothing or citing IDs that don't exist.

2.4 Self-critique — where this design can fail

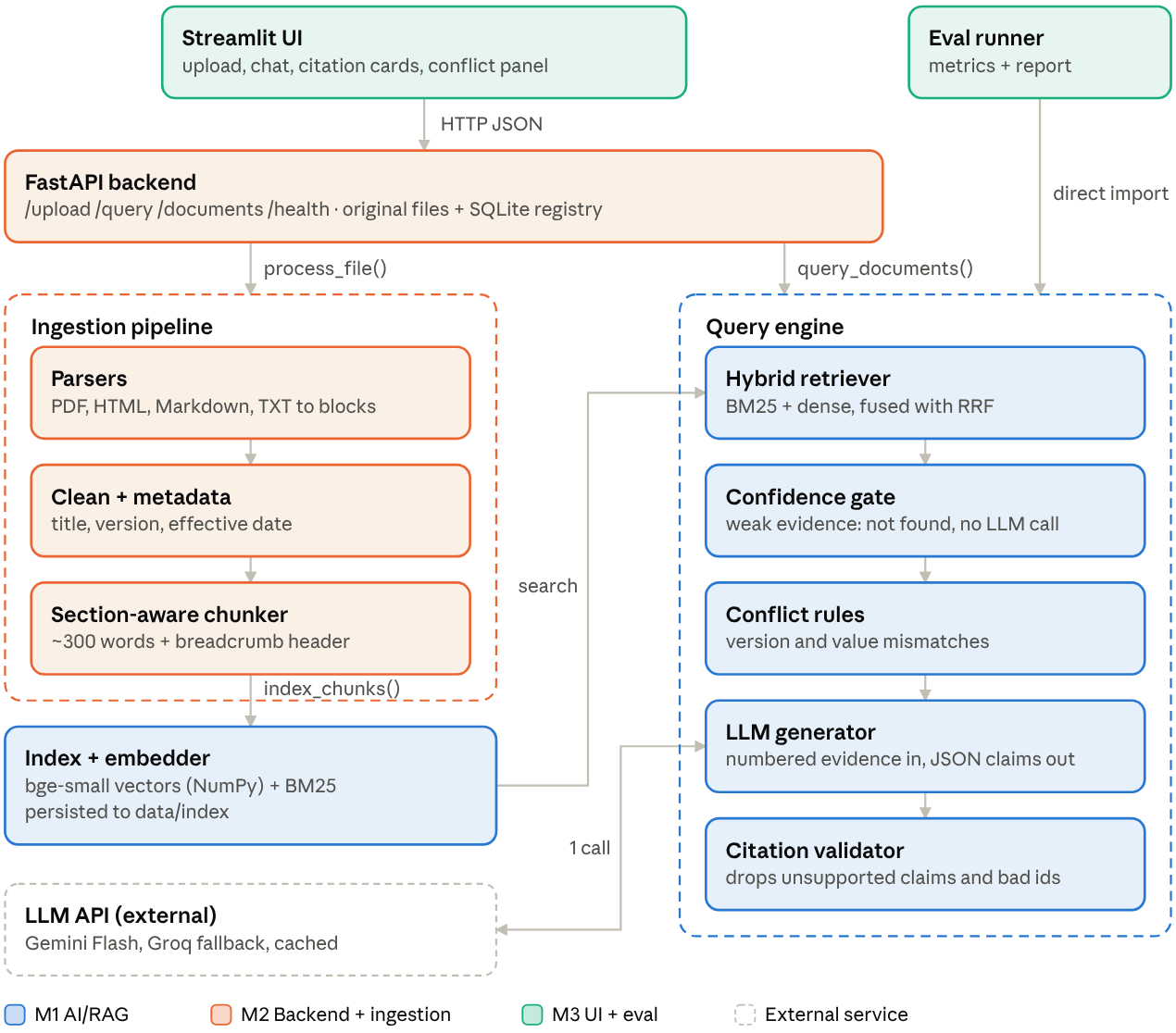
Weakness	Mitigation
Numeric-difference conflict rule fires on passages about *different* things (Pro plan limit vs Enterprise plan limit)	Only compare passages with high mutual similarity; label as "possible conflict"; let the LLM confirm or dismiss
Confidence thresholds tuned on ~30 questions may overfit	Report it honestly in limitations; use conservative thresholds; show the score so judges see calibration
PDF heading detection by font size is heuristic	Fall back to page-level sections ("Page 4") — page citations still satisfy FR-05

Weakness	Mitigation
Free-tier LLM rate limits during the demo	Response cache keyed by (question, evidence hash); second provider as fallback; pre-warm demo questions
Local embedding model download needs internet	Download in the first 30 minutes and commit the model folder path to config, not the repo

Stage 3 — Complete system architecture

Two pipelines share one store: an **ingestion pipeline** turns files into indexed, metadata-rich chunks, and a **query pipeline** turns a question into a validated, cited answer — or an honest refusal.

Two pipelines share one index; the LLM is called once, at the end



system architecture · coloured by owner

Colour shows who owns each part: Member 2 owns everything up to chunks, Member 1 owns the index and the query engine, Member 3 owns the UI and the eval runner, which imports the engine directly.

3.1 Component specifications

#	Component	What it does	Why needed	Input	Output	Talks to
1	Streamlit UI	Upload, document list, chat, citation and conflict cards	Demo surface	User actions	HTTP calls	API

#	Component	What it does	Why needed	Input	Output	Talks to
2	FastAPI app	Validates requests, orchestrates ingestion and query, maps errors	Clean contract between UI and engine	HTTP JSON / multipart	HTTP JSON	UI, ingestion, engine, registry
3	Document registry	SQLite table of documents + files on disk	List/delete docs, serve originals for citation links	DocumentRecord	Rows	API
4	Parsers (PDF/HTML/MD/TXT)	Turn a file into ordered text blocks with section path and page	FR-01	File path	`list[Block]`	Chunker
5	Metadata extractor	Title, version, effective date, doc kind from filename, front matter, first page	FR-01, feeds FR-06	File name, first blocks, user overrides	`DocMeta`	Chunker
6	Chunker	Section-aware split into ~300-word chunks with breadcrumb header	FR-02	Blocks + DocMeta	`list[Chunk]`	Engine
7	Embedder	Local sentence-embedding model	Semantic match (FR-03)	Texts	Normalised vectors	Index, retriever, validator
8	Index	Dense matrix + BM25 + chunk store, persisted to disk	Fast search, survives restart	Chunks + vectors	Searchable index	Retriever
9	Hybrid retriever	Dense top-20 + BM25 top-20, fused with RRF, optional rerank, top-k	FR-03	Question, filters	`list[ScoredChunk]`	Gate, conflicts, generator
10	Confidence gate	Scores evidence; decides answer / clarify / not-found	FR-07, stops hallucination early	Scored chunks, question	`EvidenceAssessment`	Engine
11	Conflict detector	Rules for version and value disagreement among hits	FR-06	Scored chunks	`list[ConflictCandidate]`	Generator, engine
12	LLM client	Provider wrapper, JSON mode, retry, cache, fallback provider	Isolates the only external dependency	Prompt	JSON text	Generator
13	Generator	Builds evidence-numbered prompt, parses JSON	FR-04	Question, evidence, conflict hints	`DraftAnswer`	LLM client, validator
14	Citation validator	Drops invalid or unsupported claims, picks excerpts, renumbers	FR-04, FR-05	Draft + evidence	Claims + citations	Engine
15	Engine façade	`query_documents()`, `index_chunks()` — the only AI entry points	One stable contract for API and eval	Question / chunks	`QueryResult`	API, eval runner
16	Eval runner	Runs labelled questions, computes metrics, writes report	FR-08 + report deliverable	`questions.jsonl`	Metrics JSON + Markdown report	Engine (direct import)

3.2 Data flow (end to end)

```
sequenceDiagram
    participant U as Streamlit UI
    participant A as FastAPI
    participant I as Ingestion
    participant E as RAG Engine
    participant L as LLM API
    U->>A: POST /upload (files, optional version/date)
    A->>I: process_file(path, user_meta)
    I-->>A: DocumentRecord + chunks
    A->>E: index_chunks(chunks)
    E-->>A: IndexStats
    A-->>U: upload results
    U->>A: POST /query {question}
    A->>E: query_documents(question)
    E->>E: hybrid retrieve, gate, conflict rules
    alt evidence too weak
        E-->>A: status not_found (no LLM call)
    else enough evidence
        E->>L: one JSON-mode call with numbered evidence
        L-->>E: claims + evidence ids + conflicts
        E->>E: validate citations, build result
        E-->>A: QueryResult
    end
    A-->>U: answer, citations, conflicts, confidence
```

3.3 Document ingestion workflow

```
flowchart TD
    A[File uploaded] --> B{Valid type and size?}
    B -- no --> X[400 UNSUPPORTED_FORMAT]
    B -- yes --> C[doc_id = sha256 of bytes, first 12 chars]
    C --> D{Already indexed?}
    D -- yes --> Y[Return existing record, status duplicate]
    D -- no --> E[Save original to data/uploads]
    E --> F{Format}
    F -- PDF --> P1[PyMuPDF: spans with font size and page]
    F -- HTML --> P2[BeautifulSoup: drop nav/script, walk h1-h6, p, li,
table]
    F -- MD --> P3[Line scan: # headings, front matter, keep code fences]
    F -- TXT --> P4[Heuristics: CAPS lines, numbered headings, underlines]
```

```
    P1 --> G[Blocks: text, section_path, page]
    P2 --> G
    P3 --> G
    P4 --> G
    G --> H[Clean: NFKC, whitespace, de-hyphenate, strip repeated
headers/footers]
    H --> M[Extract metadata: title, version, effective date, doc kind; user
values win]
    M --> K[Section-aware chunking ~300 words + breadcrumb header]
    K --> L[Embed embed_text, append to matrix, rebuild BM25, persist]
    L --> R[Insert registry row, return IngestResult]
```

Chunking rules, in order:

1. Never let a chunk cross a top-level section boundary.
2. Target ~250–350 words; hard max ~450. Split long sections on paragraph, then sentence boundaries, with ~40 words of overlap.
3. Merge tiny sections (< 40 words) into the next sibling under the same parent.
4. Keep tables and code blocks whole where possible.
5. Build `embed_text = "{doc_title} > {section_path}\n{text}"` — the breadcrumb header makes a chunk like "Click Reset" findable for "how to reset SSO password".

3.4 Query, retrieval, generation, citation, conflict and fallback workflow

```
flowchart TD
    Q[Question] --> N[Normalise; optional glossary expansion]
    N --> D1[Dense: cosine top 20]
    N --> B1[BM25: top 20]
    D1 --> F[Reciprocal Rank Fusion]
    B1 --> F
    F --> RR[Optional cross-encoder rerank]
    RR --> K[Top k = 5 evidence chunks]
    K --> G{Confidence gate}
    G -- top score below T_low --> NF[not_found + closest docs searched]
    G -- vague question, hits split across products -->
CL[needs_clarification + options]
    G -- pass --> C[Conflict rules: version and value mismatch]
    C --> P[Prompt: numbered evidence S1..S5 + conflict hints]
    P --> LLM{LLM call ok?}
    LLM -- error or rate limit --> EX[Extractive fallback: top passages with
citations, degraded = true]
    LLM -- ok --> V[Validate: drop bad ids, drop uncited claims, check
support]
    V --> S{Any supported claims left?}
    S -- no --> NF
    S -- yes --> R[Build QueryResult: status, claims, citations, conflicts,
confidence]
```

Retrieval (FR-03).

1. Dense: embed the question with the model's query prefix; dot product against the normalised matrix; top 20.
2. Keyword: tokenise (lowercase, keep alphanumerics and hyphens so `ERR-403` survives); BM25Okapi top 20.
3. Fuse with Reciprocal Rank Fusion: `score(d) = Σ 1 / (60 + rank_i(d))`. RRF needs no score normalisation, which is why it is the standard choice.
4. Optional rerank of the top ~15 with a cross-encoder; keep top k = 5.
5. Keep the raw dense cosine and BM25 rank of each hit — the gate and the UI's debug panel use them.

Answer generation (FR-04). One LLM call. The prompt contains only the question and evidence blocks labelled `[S1]…[S5]`, each with doc name, section, page, version and date. Rules in the system prompt: use only the evidence; every sentence cites at least one `S` id;

if the evidence disagrees, describe each side with its source and do not choose; if evidence is insufficient, return `not_found`; if the question is underspecified, return `needs_clarification` with one question. Output is JSON (`status`, `claims[]`, `conflicts[]`, `clarifying_question`, `missing_info`), requested through the provider's JSON/structured-output mode.

Citation generation (FR-05).

1. Drop any `S` id the model invented.
2. Drop claims left with no valid id; if more than a third are dropped, lower confidence one level.
3. **[R] Support check:** embed each claim and its cited chunks' sentences; if the best similarity is below a threshold, mark the claim "weakly supported" in the UI.
4. Excerpt = the cited chunk's sentence(s) most similar to the claim, ⩽ 300 characters.
5. Renumber to `[1] [2]…` in order of first use. Each citation carries doc name, section path, page (PDF), version, date, excerpt, chunk id and a link `/documents/{doc_id}/file#page=N`.

Conflict detection (FR-06) — rules propose, the LLM confirms, nothing is silently dropped.

1. **Version-sensitive rule:** two or more relevant hits come from documents of the same family (same kind or title stem) with different `version` or `effective_date`.
2. **Value-mismatch rule:** two hits from different documents are about the same thing (chunk-to-chunk cosine ⩾ ~0.75) and contain a number with the same unit but a different value ("30 days" vs "14 days", "100 requests/min" vs "500 requests/min", "$49" vs "$59").
3. Rule hits go into the prompt as hints; the LLM returns `conflicts[]` with `S` ids.
4. Merge: keep LLM conflicts with valid ids; keep any rule-detected value mismatch the LLM ignored, labelled "possible conflict".
5. Present both sides with version/date, and a hint ("The newer document, v2.1 (Mar 2025), says… — confirm which version you use"). Status becomes `conflict`.

Confidence and fallback (FR-07).

- Confidence score (0–1) = 0.6 × normalised top relevance + 0.2 × agreement (top chunk in both dense and BM25 top 5) + 0.2 × support (how many hits are close to the top score, capped at 3). Levels: high ⩾ 0.70, medium 0.45–0.70, low < 0.45. **All thresholds are placeholders — tune them on the eval set.**

- **Gate before the LLM:** top relevance below `T_low` → `not_found` without spending an API call.
- **Clarify:** question has no product/version cue *and* top hits split across different products or features with similar scores → `needs_clarification` with options drawn from metadata.
- **Low but above gate:** answer, with a visible "weak evidence" banner.
- **LLM failure:** extractive fallback — show the top passages with citations and `degraded: true`. The demo never shows a stack trace.

Status values (closed set): `answered`, `partial`, `conflict`, `needs_clarification`, `not_found`.

Stage 4 — Technology stack

Final pick: **Python 3.11 · Streamlit · FastAPI · PyMuPDF + BeautifulSoup + hand-written MD/TXT parsers · bge-small-en-v1.5 via sentence-transformers · NumPy + rank_bm25 · Gemini Flash (primary) with Groq (fallback) · SQLite + local files.** Everything except the LLM runs locally on a laptop.

4.1 Frontend

Option	Advantages	Disadvantages	Install	Time to usable	Integration
Streamlit (pick)	Pure Python; built-in chat, file uploader, expanders, columns	Re-runs script on each click; limited custom styling	`pip install streamlit`	~1.5 h for full UI	Easy — `requests` to the API
React (Vite)	Best UX and control	Build tooling, CORS, state management, JS skills; polish takes 3–4 h	Node + npm	3–4 h	Medium
Gradio	Fastest chat demo	Harder to lay out citation and conflict cards	`pip install gradio`	~1 h	Easy

4.2 Backend

Option	Advantages	Disadvantages	Install	Time	Integration
FastAPI (pick)	Pydantic validation shares the same schema file; automatic Swagger UI at `/docs` to show judges the API	Async concepts (you can write plain `def` endpoints and ignore them)	`fastapi uvicorn python-multipart`	~1 h skeleton	Easy
Flask	Very familiar, simple	No built-in validation or API docs; you hand-write schema checks	`flask`	~1 h	Easy
No backend (Streamlit imports the engine)	Fewest moving parts	No API deliverable, UI and engine coupled	—	0	Trivial

Keep the third option as your **backup plan**: if the API breaks at hour 4, Streamlit can call `query_documents()` directly. Gotcha: FastAPI file uploads silently fail without `python-multipart`.

4.3 Document extraction

Format	Pick	Why	Alternatives
PDF	**PyMuPDF (`fitz`)**	Fast; gives page numbers and font size/bold per span → heading detection	pdfplumber (better tables, slower) — add only for table-heavy PDFs; pypdf (text only, no fonts). Note PyMuPDF is AGPL — fine for a hackathon, mention it in README

Format	Pick	Why	Alternatives
HTML	**BeautifulSoup4 + lxml**	Remove `script/style/nav/header/footer`, walk `h1–h6`, `p`, `li`, `table` in order	trafilatura (great boilerplate removal, but loses heading structure control)
Markdown	**Own line parser** (~40 lines)	ATX headings `#` give the section tree directly; handle front matter and code fences	markdown-it-py (heavier than needed)
TXT	**Own heuristics**	ALL-CAPS lines, `1.2 Title` numbering, `====` / `----` underlines → headings; blank lines → paragraphs	—

Scanned (image-only) PDFs need OCR — **out of scope**; detect "no text extracted" and return a clear error.

4.4 Embedding model

Option	Advantages	Disadvantages	Install / time
BAAI/bge-small-en-v1.5 (pick)	Strong retrieval quality for its size; 384-dim; runs on CPU	~130 MB download; needs the query prefix "Represent this sentence for searching relevant passages: "	`sentence-transformers` (pulls PyTorch — install the CPU wheel early)
all-MiniLM-L6-v2	Smaller, faster, very well known	Noticeably weaker on retrieval	Same
Gemini Embedding API	No local model	Uses API quota at index time; network latency; another failure point	API key
fastembed (ONNX)	Same bge model without PyTorch; light install	Less familiar	`pip install fastembed` — use if the PyTorch install is slow

4.5 LLM for answer generation

A chat subscription (Claude Pro, Gemini Advanced, ChatGPT Plus) does **not** give API access. You need an API key from the provider's developer console, with free-tier quota or credits.

Option	Access	Advantages	Disadvantages
Gemini Flash via Google AI Studio (primary)	Free API key; Flash models are on the free tier	JSON output with a response schema; large context; good quality	Free-tier limits are per project and shown only in your AI Studio rate-limit page; reports range from ~20 to ~1,000 requests/day depending on model. Free-tier prompts may be used to improve Google products — use non-sensitive documents
Groq (fallback)	Free plan	Very fast; JSON mode; open models (e.g. gpt-oss-120b, Qwen)	Sample free limits ~30 requests/min, 1,000/day, but only **8,000 tokens/min** — about 3 questions/min with a 2,500-token prompt
Claude API	Anthropic Console account with prepaid credits	Excellent instruction following and grounding	No free tier to rely on; separate from any Claude.ai subscription
Ollama (local 3–8B model)	Free, offline	No quota	Slow on CPU laptops; weaker JSON compliance — emergency only

Design consequence: the LLM client must support **two providers behind one function**, a **disk cache** keyed by (prompt hash), and a **retry with backoff**. Check each key with a one-line test call in the first 30 minutes — and check your real quota on the provider's limits page then, not during the demo. Sources: Gemini free tier, 2026, Groq free tier.

4.6 Retrieval and storage

Option	Advantages	Disadvantages	Install	Verdict
NumPy matrix (dense)	Exact search, zero setup, milliseconds at < 10k chunks	Linear scan won't scale to millions	none	**Pick**
FAISS (`IndexFlatIP`)	Same results, scales further	Extra wheel; adds nothing at this size	`faiss-cpu`	Optional swap
Chroma	Persistence and filters built in	Own embedding coupling, extra API to learn, version churn	`chromadb`	Skip
rank_bm25	Pure Python BM25Okapi, tiny	Rebuild on each ingest (milliseconds here)	`rank_bm25`	**Pick**
Hybrid via RRF	No score normalisation; robust	One extra function	none	**Pick**
Cross-encoder `ms-marco-MiniLM-L-6-v2`	Better top-3 ordering; cleaner confidence	~90 MB, adds latency on CPU	via sentence-transformers	Optional, phase 2
SQLite (stdlib)	Document registry and query log; zero install	Single writer (fine here)	none	**Pick**
Local files	Originals in `data/uploads`, index in `data/index` (`chunks.jsonl`, `embeddings.npy`)	Not shared across machines	none	**Pick**

4.7 Final stack and install list

Layer	Choice
Language	Python 3.11, one virtual environment, one `requirements.txt`
UI	Streamlit
API	FastAPI + Uvicorn + python-multipart
Schemas	Pydantic v2 (one shared `schemas.py`)
Parsing	PyMuPDF, beautifulsoup4 + lxml, own MD/TXT parsers
Embeddings	sentence-transformers + `BAAI/bge-small-en-v1.5`
Keyword	rank_bm25
Dense search	NumPy
Reranker [O]	`cross-encoder/ms-marco-MiniLM-L-6-v2`
LLM	Gemini Flash (google-genai SDK) primary; Groq fallback
Storage	SQLite + local folders
Config	python-dotenv, `.env` (never commit keys)
Tests	pytest

Total install time is about 10–15 minutes, most of it PyTorch and the model download. Do it in the first half hour on every laptop.

Stage 5 — Dividing the work

Change one thing in your proposed split: **move document processing and chunking from Member 1 to Member 2.** Otherwise Member 1 sits both upstream and downstream of everyone and becomes the bottleneck, while Member 2 runs out of work after the FastAPI skeleton (~1.5 h).

5.1 Critique of the initial allocation

Problem in the initial split	Effect	Fix
Member 1 owns ingestion **and** retrieval **and** generation **and** conflicts **and** confidence	~8 h of work in a 5 h window; every other member waits on them	Ingestion (parsers, metadata, chunker) moves to Member 2
Member 2 owns only a thin API	Idle by hour 2	Member 2 owns everything from "file arrives" to "list of Chunk objects"
Evaluation needs a corpus and labels before anything can be measured	Eval would start too late to tune thresholds	Member 3 builds the corpus **first**, in hour 1
Nobody owns the shared data shapes	Integration breaks at hour 3	One frozen `schemas.py`, written together in the first 30 min; Member 1 is its gatekeeper
UI waits on a working engine	Frontend blocked	Member 1 ships a stub engine that returns a canned `QueryResult` by 0:30

5.2 Revised roles

- **Member 1 — AI/RAG engineer (you):** everything between "here are chunks" and "here is a validated QueryResult": embeddings, index, hybrid retrieval, confidence gate, conflict rules, LLM client, prompt, generation, citation validation, engine façade, threshold tuning.
- **Member 2 — Backend and ingestion engineer:** everything between "a file arrives over HTTP" and "here are chunks": FastAPI, upload validation, storage, registry, four parsers, cleaning, metadata extraction, chunker, error handling, end-to-end smoke test.
- **Member 3 — Frontend, evaluation and demo engineer:** corpus, labelled question set, Streamlit UI, eval runner and metrics, evaluation report, README, demo script and backup recording.

5.3 Responsibility matrix

	Member 1 — AI/RAG	Member 2 — Backend + ingestion	Member 3 — Frontend + eval + demo
Components	Embedder, index, retriever, (reranker), confidence gate, conflict detector, LLM client, generator, citation validator, engine façade	FastAPI app, registry, file storage, parsers ×4, cleaner, metadata extractor, chunker, ingestion pipeline	Streamlit UI, API client, sample corpus, labelled questions, eval runner, metrics, eval report, README, demo
Owns files	`rag/*`, `config.py`, gatekeeper of `common/schemas.py`	`backend/*`, `ingestion/*`, `tests/test_ingestion.py`, `tests/test_api.py`	`frontend/*`, `eval/*`, `README.md`, `docs/*`
Tech	sentence-transformers, NumPy, rank_bm25, google-genai / groq SDK, Pydantic	FastAPI, Uvicorn, PyMuPDF, BeautifulSoup, SQLite, pytest	Streamlit, requests, pandas (metrics tables)
Hands to others	`index_chunks()`, `query_documents()`, `retrieve()`	`process_file()` → chunks; HTTP API	Corpus + questions (to M1, M2); UI; metrics that drive M1's tuning
Needs from others	Chunks (MD/TXT parser from M2 by ~1:00; corpus from M3 by ~1:00)	`index_chunks`, `query_documents` signatures (stub at 0:30)	Running API (stub by 0:45); `retrieve()` for retrieval eval

	Member 1 — AI/RAG	Member 2 — Backend + ingestion	Member 3 — Frontend + eval + demo
Tests	`tests/test_retrieval.py`: known question → known chunk in top 3; gate refuses a nonsense question; validator drops fake ids	Parser test per format; chunker never crosses top-level sections; API TestClient for upload/query/errors	Eval run is the system test; manual UI checklist; demo dry-runs
Must not modify	`backend/`, `ingestion/`, `frontend/`, `eval/`	`rag/`, `frontend/`, `eval/`	`rag/`, `backend/`, `ingestion/`

`common/schemas.py` changes need agreement from at least two members and must be **additive** (new optional fields only) after 0:30.

5.4 Member 1 — tasks in order

1. Co-write `common/schemas.py` and `config.py`; ship stub `rag/engine.py` returning a canned `QueryResult` (by 0:30).
2. `embeddings.py` — load model once; `encode(texts, is_query)` with the bge query prefix; normalised vectors.
3. `index.py` — add/remove chunks, NumPy matrix, BM25 rebuild, save/load to `data/index/`.
4. `retriever.py` — dense top-20, BM25 top-20, RRF, metadata filters, top-k. Test on M3's corpus using M2's MD/TXT parser.
5. `llm.py` — `complete_json(system, user, schema)`; Gemini primary, Groq fallback, disk cache, retry with backoff.
6. `prompts.py` + `generator.py` — evidence block formatting, JSON parsing, defensive repair of bad JSON.
7. `validator.py` — drop invalid ids and uncited claims, choose excerpts, renumber citations.
8. `confidence.py` — score, levels, gate, clarification rule.
9. `conflicts.py` — version rule, value-mismatch rule, merge with LLM conflicts.
10. Replace the stub in `engine.py`; run M3's eval; tune `T_low` and level cut-offs.
11. **[O]** Cross-encoder rerank behind a config flag; **[O]** claim-support check.

Deliverables: working `query_documents()`, `index_chunks()`, `retrieve()`, `delete_document()`; tuned thresholds in `config.py`; 3–4 retrieval unit tests.

5.5 Member 2 — tasks in order

1. Co-write schemas; FastAPI skeleton with `/health`, `/upload`, `/documents`, `/query` wired to the stub engine (by 0:45) so M3 can call a real API.
2. `storage.py` — save originals, SQLite registry, duplicate detection by content hash.
3. Markdown and TXT parsers **first** (hand to M1 by ~1:00), then HTML, then PDF.
4. `cleaner` and `metadata.py` — title, version, effective date, doc kind; user-supplied values override.
5. `chunker.py` — section-aware rules from 3.3.
6. `ingestion/pipeline.py` — `process_file(path, user_meta) -> (DocumentRecord, list[Chunk])`.
7. Wire `/upload` → `process_file` → `engine.index_chunks`; add `DELETE /documents/{id}`, `GET /documents/{id}/file`, `POST /reset`.
8. Error schema, exception handlers, file size/type limits, LLM timeout mapping.
9. Tests: one fixture file per format, API tests.
10. From ~3:30: **integration owner** — runs the end-to-end smoke script after every merge.

Deliverables: running API with Swagger docs at `/docs`; four parsers; chunker; tests.

5.6 Member 3 — tasks in order

1. **Corpus first** (by 1:00): 8–10 short documents across all four formats, with planted conflicts, version differences and deliberate gaps (see Stage 9).
2. Streamlit UI against a mock JSON response (same shape as `QueryResult`) — upload panel, document list, chat, citation cards, conflict panel, confidence badge.
3. `eval/questions.jsonl` — ~30 labelled questions, six categories.
4. `eval/run_eval.py` — retrieval metrics first (needs only `retrieve()`), then end-to-end status, citation and fallback metrics.
5. Point the UI at the real API; loading spinners and error messages.
6. Auto-generate `eval/reports/eval_report.md`; write README.
7. Demo script, pre-warm demo questions, screenshots and a backup screen recording.

Deliverables: corpus, labelled set, UI, eval report, README, 5+ evidence-backed examples, demo script.

5.7 Git workflow (keeps merges painless)

- One repo, `main` branch, folder ownership instead of long-lived branches. Each member commits only inside their folders.
- Pull and push at every checkpoint (0:30, 1:30, 2:15, 3:30, 4:15).
- `data/`, `.env`, model caches in `.gitignore`.

Stage 6 — API contracts and integration

Three contracts let the three members work without touching each other's code: the **HTTP API** (M2 ↔ M3), the **ingestion function** (M2 → M1), and the **engine functions** (M1 → M2 and M3). All shapes live in one file, `common/schemas.py`, as Pydantic models.

6.1 HTTP endpoints

Method + path	Purpose	Success	Errors
`GET /health`	Liveness, index size, active LLM provider	200	—
`POST /upload`	Multipart `files[]` + optional `version`, `effective_date`, `product`, `doc_kind`	200 with per-file status	413, 415, 422 per file inside results
`GET /documents`	List indexed documents	200	—
`GET /documents/{doc_id}/file`	Serve the original — target of citation links	200	404
`DELETE /documents/{doc_id}`	Remove from registry and index	200	404
`POST /query`	Ask a question	200 (also for `not_found` and degraded answers)	409 index empty, 422 bad request
`POST /reset`	Clear everything (demo convenience)	200	—

Method + path	Purpose	Success	Errors
`GET /documents/{doc_id}/chunks` **[O]**	Show how a document was chunked	200	404

Rule: "not found" and "LLM unavailable" are **answers, not errors** — they return 200 with the right `status` or `degraded: true`. HTTP errors are only for bad input or missing resources.

6.2 Shared data shapes

DocumentRecord (metadata structure)

```
{
  "doc_id": "a1b2c3d4e5f6",
  "filename": "billing_policy_v2.pdf",
  "title": "Billing Policy",
  "doc_type": "pdf",
  "doc_kind": "policy",
  "version": "2.0",
  "effective_date": "2025-03-01",
  "product": null,
  "pages": 12,
  "num_chunks": 34,
  "uploaded_at": "2026-10-31T10:12:00Z",
  "source_url": "/documents/a1b2c3d4e5f6/file"
}
```

`doc_type` ∈ `pdf | html | md | txt`. `doc_kind` ∈ `manual | faq | release_notes | policy | support_article | other`. `doc_id` = first 12 hex chars of SHA-256 of the file bytes (gives free duplicate detection).

Chunk

```
{
  "chunk_id": "a1b2c3d4e5f6:0007",
  "doc_id": "a1b2c3d4e5f6",
  "doc_title": "Billing Policy",
  "doc_type": "pdf",
  "doc_kind": "policy",
  "section_path": ["Refunds", "Annual plans"],
  "page_start": 4,
  "page_end": 4,
  "ordinal": 7,
  "text": "Annual subscriptions may be refunded within 30 days of
purchase...",
  "embed_text": "Billing Policy > Refunds > Annual plans\nAnnual
subscriptions may be refunded...",
  "version": "2.0",
  "effective_date": "2025-03-01",
  "word_count": 212
}
```

`page_start` / `page_end` are `null` for non-PDF files. Version and date are copied onto every chunk so retrieval and conflict rules never need a registry lookup.

QueryRequest

```
{
  "question": "How many days do I have to get a refund on an annual plan?",
  "top_k": 5,
  "filters": {"doc_ids": null, "version": null, "product": null},
  "debug": false
}
```

QueryResult — conflict example (citation, conflict and confidence structures)

```
{
  "query_id": "q_8f3a21",
  "question": "How many days do I have to get a refund on an annual plan?",
  "status": "conflict",
  "answer": "The documents disagree. Billing Policy v2.0 (effective 2025-03-
01) allows refunds within 30 days of purchase [1]. The Billing FAQ (updated
2023-06-10) says 14 days [2].",
  "claims": [
    {"text": "Billing Policy v2.0 allows refunds within 30 days of
purchase.", "citation_ids": [1], "support": "supported"},
```

```
    {"text": "The Billing FAQ says 14 days.", "citation_ids": [2], "support":
"supported"}
  ],
  "citations": [
    {
      "id": 1,
      "chunk_id": "a1b2c3d4e5f6:0007",
      "doc_id": "a1b2c3d4e5f6",
      "doc_title": "Billing Policy",
      "filename": "billing_policy_v2.pdf",
      "section": "Refunds > Annual plans",
      "page": 4,
      "version": "2.0",
      "effective_date": "2025-03-01",
      "excerpt": "Annual subscriptions may be refunded within 30 days of
purchase.",
      "source_url": "/documents/a1b2c3d4e5f6/file#page=4",
      "relevance": 0.83
    },
    {
      "id": 2,
      "chunk_id": "9e8d7c6b5a40:0003",
      "doc_id": "9e8d7c6b5a40",
      "doc_title": "Billing FAQ",
      "filename": "billing_faq.md",
      "section": "Refunds",
      "page": null,
      "version": null,
      "effective_date": "2023-06-10",
      "excerpt": "You can request a refund within 14 days of your annual
renewal.",
      "source_url": "/documents/9e8d7c6b5a40/file",
      "relevance": 0.79
    }
  ],
  "conflicts": [
    {
      "type": "value_mismatch",
      "topic": "Refund window for annual plans",
      "summary": "30 days in Billing Policy v2.0 vs 14 days in the Billing
FAQ",
      "positions": [
        {"citation_id": 1, "statement": "30 days", "version": "2.0",
```

```
"effective_date": "2025-03-01"},
        {"citation_id": 2, "statement": "14 days", "version": null,
"effective_date": "2023-06-10"}
      ],
      "resolution_hint": "Billing Policy v2.0 is the newer document; confirm
which policy applies to your account.",
      "detected_by": "rules+llm"
    }
  ],
  "confidence": {"level": "medium", "score": 0.62, "reasons": ["Strong
matches in 2 documents", "Sources disagree"]},
  "clarifying_question": null,
  "degraded": false,
  "model": "gemini-flash",
  "timings_ms": {"retrieval": 42, "generation": 1830, "total": 1910},
  "debug": null
}
```


Field rules:

- `status` ∈ `answered | partial | conflict | needs_clarification | not_found`.
- `claims[].support` ∈ `supported | weak` (claims with no valid citation are removed, never shown).
- `conflicts[].type` ∈ `version_sensitive | value_mismatch | contradiction`; `detected_by` ∈ `rules | llm | rules+llm`.
- `confidence.level` ∈ `high | medium | low`; `score` is 0–1; `reasons` are short human-readable strings shown in the UI.
- `debug` (only when `debug: true`): retrieved chunks with `dense_score`, `bm25_rank`, `dense_rank`, `rrf_score`, optional `rerank_score`, and the gate decision.

not_found example (no LLM call made)

```
{
  "query_id": "q_51c0aa",
  "question": "What is the parental leave policy?",
  "status": "not_found",
  "answer": "I couldn't find information about this in the uploaded
documents. The closest documents searched were the Billing Policy and the
Admin Guide, which don't cover leave policies.",
  "claims": [],
  "citations": [],
  "conflicts": [],
  "confidence": {"level": "low", "score": 0.12, "reasons": ["Best match
relevance 0.31 is below threshold 0.45"]},
  "clarifying_question": null,
  "degraded": false,
  "model": null,
  "timings_ms": {"retrieval": 38, "generation": 0, "total": 40},
  "debug": null
}
```


needs_clarification example: same shape, `status: "needs_clarification"`, `clarifying_question: "Which limit do you mean — API requests per minute, storage per workspace, or users per plan?"`, and citations for the candidate passages so the user can see why.

Upload response


```
{
  "results": [
    {"filename": "billing_policy_v2.pdf", "status": "indexed", "document":
{"doc_id": "a1b2c3d4e5f6", "title": "Billing Policy", "num_chunks": 34},
"error": null},
    {"filename": "scan.pdf", "status": "failed", "document": null, "error":
{"code": "EMPTY_DOCUMENT", "message": "No extractable text. Scanned PDFs
(images) are not supported."}}
  ],
  "index": {"documents": 9, "chunks": 312}
}
```


Per-file `status` ∈ `indexed | duplicate | failed`. A batch with one bad file still indexes the good ones.

Error response (for HTTP 4xx/5xx)

```
{"error": {"code": "INDEX_EMPTY", "message": "Upload at least one document before asking questions.", "details": {}}}
```

Code	HTTP	When
`UNSUPPORTED_FORMAT`	415	Extension not pdf/html/htm/md/markdown/txt
`FILE_TOO_LARGE`	413	Over 20 MB
`EMPTY_DOCUMENT`	422	No text extracted (scanned PDF, empty file)
`PARSE_FAILED`	422	Parser exception (corrupt file, bad encoding)
`INVALID_REQUEST`	422	Empty question or over 1,000 characters
`DOCUMENT_NOT_FOUND`	404	Unknown `doc_id`
`INDEX_EMPTY`	409	Query before any upload
`INTERNAL_ERROR`	500	Anything else (logged with `query_id`)

6.3 Python interfaces between modules

Your suggested functions are kept but split by owner: `ingest_document` becomes M2's `process_file` plus M1's `index_chunks`; `retrieve_chunks` becomes `retrieve`; `generate_answer` and `detect_conflicts` stay internal to M1 behind `query_documents`.

M2 → M1: ingestion output

Function	Input	Output	Raises
`ingestion.process_file(path, user_meta=None)`	File path; optional `UserMeta` (version, effective_date, product, doc_kind)	`(DocumentRecord, list[Chunk])`	`IngestionError(code, message)`

M1 public engine API (used by M2's API and M3's eval)

Function	Input	Output	Notes
`engine.index_chunks(doc, chunks)`	`DocumentRecord`, `list[Chunk]`	`IndexStats`	Embeds, appends, rebuilds BM25, persists
`engine.delete_document(doc_id)`	`str`	`IndexStats`	
`engine.retrieve(question, k=5, filters=None)`	`str`, `int`, `Filters`	`list[ScoredChunk]`	No LLM. Used by retrieval eval
`engine.query_documents(question, top_k=5, filters=None, debug=False)`	as above	`QueryResult`	Never raises for LLM problems; returns `degraded: true`
`engine.index_stats()` / `engine.reset()`	—	`IndexStats`	

M1 internal steps (inside `query_documents`)

Function	Input	Output
`assess_evidence(question, hits)`	question, `list[ScoredChunk]`	`EvidenceAssessment` {decision: `answer | clarify | not_found`, confidence, reasons, clarify_options}
`detect_conflicts(hits)`	`list[ScoredChunk]`	`list[ConflictCandidate]`
`generate_answer(question, hits, conflict_hints)`		`DraftAnswer` (raw LLM JSON, parsed)
`validate_answer(draft, hits, conflict_candidates)`		claims, citations, conflicts, confidence adjustment

6.4 How each member works independently on day one

- **M3** builds the UI against `eval/fixtures/sample_query_result.json` (the conflict example above) and the stub API — no engine needed.
- **M2** builds endpoints against the stub engine (`query_documents` returns the fixture; `index_chunks` just counts).

- **M1** builds retrieval against chunks produced by M2's MD/TXT parser on M3's corpus — no API needed. Until those land (~1:00), a 10-line throwaway paragraph splitter in a scratch notebook is enough.

Stage 7 — Project folder structure

Five top-level code folders, one per ownership boundary, plus one shared schema file. About 25 Python files in total — no service layers, repositories or dependency-injection frameworks.

```
enterprise-docqa/
├── README.md                       # M3  setup, formats, approach, trade-offs
├── requirements.txt                # M2  (everyone adds, M2 merges)
├── .env.example                    # M1  GEMINI_API_KEY, GROQ_API_KEY,
LLM_PROVIDER
├── .gitignore                      # M2  data/, .env, __pycache__, model
caches
├── config.py                       # M1  model names, k values, thresholds,
paths
├── common/
│   └── schemas.py                  # ALL (M1 gatekeeper) — frozen at 0:30,
additive changes only
├── ingestion/                      # M2
│   ├── parsers.py                  # parse_pdf, parse_html, parse_md,
parse_txt -> list[Block]
│   ├── cleaner.py                  # normalise text, strip repeated
headers/footers
│   ├── metadata.py                 # title, version, effective_date, doc_kind
│   ├── chunker.py                  # section-aware chunking
│   └── pipeline.py                 # process_file(path, user_meta)
├── rag/                            # M1
│   ├── embeddings.py               # model load + encode
│   ├── index.py                    # NumPy matrix, BM25, persistence
│   ├── retriever.py                # dense, bm25, RRF, (rerank), filters
│   ├── confidence.py               # score, levels, gate, clarify rule
│   ├── conflicts.py                # version + value-mismatch rules, merge
│   ├── llm.py                      # provider wrapper, JSON mode, cache,
retry, fallback
│   ├── prompts.py                  # system prompt + evidence formatting
│   ├── generator.py                # build prompt, call LLM, parse JSON
│   ├── validator.py                # citation validation, excerpts,
renumbering
```

```
|   └── engine.py                  # PUBLIC: index_chunks, retrieve,
query_documents, delete_document
├── backend/                       # M2
|   ├── main.py                    # FastAPI app + all routes (one file is
fine)
|   ├── storage.py                 # SQLite registry + file save
|   └── errors.py                  # error codes, exception handlers
├── frontend/                      # M3
|   ├── app.py                     # Streamlit UI
|   └── api_client.py              # requests wrapper; MOCK mode reads
fixture JSON
├── eval/                          # M3
|   ├── corpus/                    # sample documents (all 4 formats)
|   ├── questions.jsonl            # labelled question set
|   ├── fixtures/sample_query_result.json
|   ├── metrics.py                 # recall@k, precision@k, MRR, status
accuracy
|   ├── run_eval.py                # runs everything, writes report
|   └── reports/eval_report.md     # generated + hand-written analysis
├── tests/
|   ├── fixtures/                  # M2  one tiny file per format
|   ├── test_ingestion.py          # M2
|   ├── test_api.py                # M2
|   └── test_retrieval.py          # M1
├── scripts/
|   ├── smoke_test.py              # M2  upload corpus -> ask 5 questions ->
assert statuses
|   └── check_llm.py               # M1  one-line test call per provider
├── docs/
|   ├── architecture.md            # M3 (diagrams from this doc)
|   └── demo_script.md             # M3
└── data/                          # git-ignored, created at runtime
    ├── uploads/
    ├── index/                     # chunks.jsonl, embeddings.npy, meta.json
    ├── llm_cache/                 # one JSON file per prompt hash
    └── app.db                     # SQLite registry + query log
```

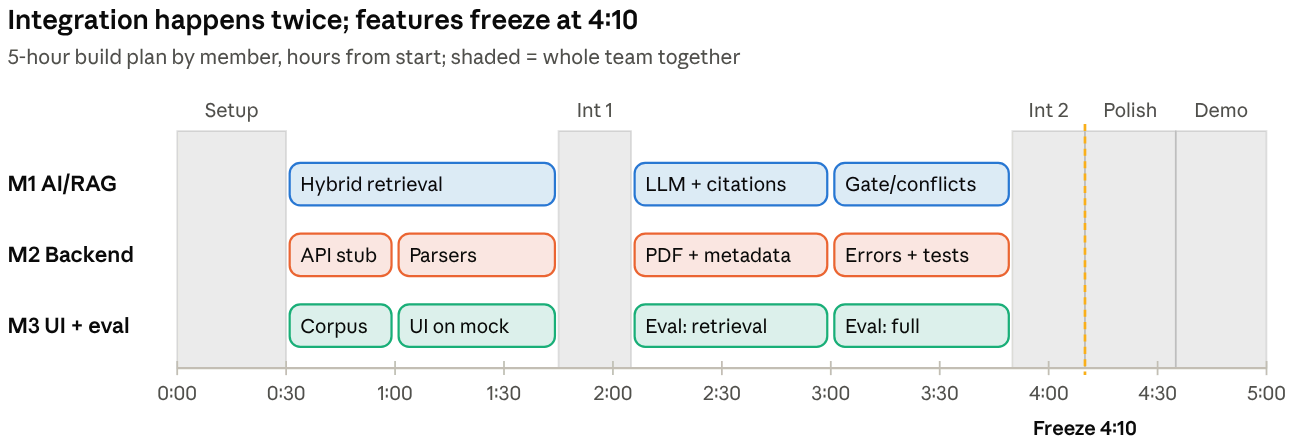
Decisions worth defending:

- `rag/engine.py` is the **only** file anyone outside M1 imports from `rag/`. M1 can restructure everything behind it freely.
- `ingestion/pipeline.py` is the only file M1 needs from `ingestion/`.

- The API runs the engine **in-process** (a module import, not a second service). One process to start, no network hop, no serialization bugs.
- Run commands: `uvicorn backend.main:app --reload --port 8000` and `streamlit run frontend/app.py`. Run both from the repo root so imports resolve.

Stage 8 — Execution plan (5 hours, with an 8-hour variant)

Coding gets about 3 h 20 min; setup, two integrations, polish and demo rehearsal get the other 1 h 40 min. Feature freeze is at **4:10** no matter what.



5-hour build timeline · 3 members, 2 integrations

Each member builds alone between the shaded team blocks; nothing new starts after the freeze line.

8.1 Phase-by-phase plan

Time	Member 1 — AI/RAG	Member 2 — Backend + ingestion	Member 3 — UI + eval + demo	Gate to pass
0:00–0:30 **Setup (all)**	Repo, venv, install, download bge model, test both LLM keys with `check_llm.py`; lead writing `schemas.py`; push stub `engine.py`	Repo skeleton, `.gitignore`, `requirements.txt`; co-write schemas	Co-write schemas; write `sample_query_result.json`; outline corpus	**A (0:30):** schemas frozen, stub engine importable, all keys work

Time	Member 1 — AI/RAG	Member 2 — Backend + ingestion	Member 3 — UI + eval + demo	Gate to pass
0:30–1:45 **Build 1**	`embeddings.py`, `index.py`, `retriever.py` (dense + BM25 + RRF); test on corpus	API with stub engine (0:45); `storage.py`; MD + TXT parsers (1:00); chunker; HTML parser	Corpus finished (1:00); Streamlit UI on mock data; draft `questions.jsonl`	**B (1:45):** retrieval returns sensible top-5 for 5 corpus questions
1:45–2:05 **Integration 1 (all)**	Make stub `query_documents` return real retrieved passages (extractive mode)	Wire `/upload` → `process_file` → `index_chunks`	Point UI at real API	Upload corpus through UI, see real passages as citations
2:05–3:00 **Build 2**	`llm.py`, `prompts.py`, `generator.py`, `validator.py`	PDF parser, metadata (version/date), error handlers	`run_eval.py` retrieval metrics (Recall@k, MRR) on real `retrieve()`; finish labels	**C (3:00):** cited LLM answers end to end; first Recall@5 number
3:00–3:50 **Build 3**	`confidence.py` gate, `conflicts.py`; tune thresholds from M3's numbers	Delete/file/reset endpoints, tests, `smoke_test.py`; becomes integration owner	End-to-end eval (status, citation, fallback accuracy); conflict and confidence UI	Conflict and not-found questions behave correctly
3:50–4:10 **Integration 2 (all)**	Fix what the full eval exposes	Run smoke test after each merge	Run full eval, save report	**Feature freeze at 4:10**
4:10–4:35 **Polish**	Write retrieval/answer method section of report; pre-warm LLM cache with demo questions	README setup, formats, API sections	Eval report, README overview, demo script, screenshots	All deliverables committed
4:35–5:00 **Demo (all)**	Rehearse technical Q&A	Rehearse; keep a terminal with Swagger ready	Run demo twice; record backup video	Clean final push

8.2 What must exist before each integration

- **Before Integration 1:** `process_file` works for MD and TXT; `index_chunks` and `retrieve` work; API upload route exists; UI can upload and render a `QueryResult`.
- **Before Integration 2:** LLM generation with validated citations; gate; at least the version conflict rule; PDF and HTML parsing; eval runner producing numbers.

8.3 If you fall behind — cut in this order

1. Cross-encoder reranker (only start it if you are ahead at 3:00).
2. Claim-support similarity check.
3. LLM conflict confirmation — keep rules only.
4. Rule-based clarification — rely on the LLM's `needs_clarification`.
5. HTML table handling and TXT heading heuristics — paragraphs only.
6. `DELETE`, `/reset`, filters, chunk viewer.
7. UI polish.

Never cut: claim citations, the not-found gate, the version-conflict rule, the eval with Recall@k and MRR, the README, five evidence-backed examples.

Escalation triggers:

- Retrieval not working at checkpoint B → M2 pairs with M1 for 20 minutes; M2's PDF work slides.
- API still broken at 3:00 → Streamlit imports `engine` directly (backup path); M2 keeps fixing the API in parallel.
- LLM quota exhausted → switch `LLM_PROVIDER`; demo from cache; extractive mode as last resort.

8.4 If you actually have 8 hours

Keep the first 3:50 identical, then add three hours **before** the freeze, and keep the last hour for polish and demo:

Extra hour	M1	M2	M3
+1	Cross-encoder rerank behind a flag	Chunk viewer endpoint; PDF table handling	Retrieval ablation table (BM25 vs dense vs hybrid vs hybrid+rerank)

Extra hour	M1	M2	M3
+2	Claim-support check; clarification rule	Query log in SQLite; feedback thumbs	Grow labelled set to 50; LLM-as-judge groundedness on a sample
+3	Threshold sweep; failure-case fixes	Hardening, load-test 20 queries	Document viewer that highlights cited excerpts; failure analysis write-up

Stage 9 — Evaluation and differentiators

The evaluation is where you win: plant known conflicts and gaps in a corpus you control, label ~30 questions against **evidence text rather than chunk IDs**, and report retrieval, status, citation and conflict metrics plus a retrieval ablation.

9.1 Synthetic corpus design (if organisers supply none)

Use a clearly fictional product, e.g. "Nimbus Workspace". 8–10 short documents (1–4 pages each), all four formats, with planted test cases:

File	Format	Kind	Planted content
`admin_guide_v3.pdf`	PDF	manual, v3.0, 2025	SSO/SAML setup, roles, password reset, API keys — multi-page so page citations show
`api_reference_v2.html`	HTML	manual, v2.4, 2023	Rate limit **100 requests/min**
`release_notes_3_0.md`	MD	release notes, v3.0, 2025	Rate limit raised to **300 requests/min**; basic auth deprecated → version conflict
`billing_policy_v2.pdf`	PDF	policy, v2.0, effective 2025-03-01	Annual plan refunds within **30 days**

File	Format	Kind	Planted content
`billing_faq.md`	MD	FAQ, updated 2023-06-10	Refunds within **14 days** → value conflict
`troubleshooting.txt`	TXT	support article	ERR-401, ERR-403, ERR-429 causes and fixes — exact-token retrieval test
`usage_limits.html`	HTML	policy	Storage and seats per plan (Free / Pro / Enterprise) — ambiguity test ("what's the limit?")
`security_faq.md`	MD	FAQ	MFA, data retention **90 days**
`data_retention_policy.txt`	TXT	policy, 2025	Retention **180 days** for Enterprise → scoped conflict (different plan, tests false positives)
`onboarding_guide.txt`	TXT	manual	Invites, workspaces ("tenant" used as a synonym)

Deliberate gaps (unanswerable): parental leave, on-premise pricing, mobile app offline mode, GDPR DPA signing process. Document all of this in the report, as the brief requires.

9.2 Labelled question set

Format, one JSON object per line in `eval/questions.jsonl`:

```
{"id": "q14", "category": "conflict", "question": "What's the API rate limit?", "expected_status": "conflict", "gold": [{"doc": "api_reference_v2.html", "evidence": "100 requests per minute"}, {"doc": "release_notes_3_0.md", "evidence": "300 requests per minute"}], "answer_keywords": ["100", "300"], "notes": "version-sensitive: v2.4 vs v3.0"}
```

A retrieved chunk counts as **relevant** if its file matches `gold.doc` **and** its text contains `gold.evidence` (case-insensitive). This survives any change to chunk size or IDs — the single most important labelling decision.

Target mix (~30 questions) with examples:

Category	Count	Example question	Expected status	Gold evidence
Direct	8	"How do I reset a user's password?"	answered	admin_guide_v3.pdf → password reset steps
Direct		"What does error ERR-429 mean?"	answered	troubleshooting.txt → "too many requests"
Paraphrase	6	"My teammate can't get into their account, what should an admin do?"	answered	admin_guide_v3.pdf → password reset
Paraphrase		"How long until deleted data is gone for good?"	answered	security_faq.md → "90 days"
Terminology	4	"How do I configure single sign-on?" (doc says SAML/SSO)	answered	admin_guide_v3.pdf → SAML setup
Terminology		"How many seats does Pro include?" (doc says "users")	answered	usage_limits.html → Pro users
Conflict	4	"What's the refund window for annual plans?"	conflict	billing_policy_v2.pdf "30 days" + billing_faq.md "14 days"
Conflict		"What's the API rate limit?"	conflict	api_reference_v2.html "100" + release_notes_3_0.md "300"

Category	Count	Example question	Expected status	Gold evidence
Ambiguous	4	"What's the limit?"	needs_clarification	usage_limits.html, api_reference_v2.html
Ambiguous		"How do I reset it?"	needs_clarification	—
Unanswerable	4	"What is the parental leave policy?"	not_found	none
Unanswerable		"How much does the on-premise edition cost?"	not_found	none

Also include one **scoped non-conflict** ("How long is data retained on the Free plan?") to check that the 90-vs-180-day difference across plans is *not* flagged as a conflict.

How the labels were built (put this in the report): questions written by Member 3 from the corpus; paraphrase and terminology questions deliberately avoid the document's wording; gold evidence strings copied verbatim from documents; labels reviewed by a second member.

9.3 Metrics

Retrieval (on answerable questions: direct, paraphrase, terminology, conflict):

Metric	Definition	Why
Recall@k (k = 1, 3, 5)	Share of gold evidence items found in the top k	For conflict questions this checks that **both** sides were retrieved
Hit@k	Share of questions with at least one relevant chunk in the top k	Easy for judges to read
MRR	Mean of 1 / rank of the first relevant chunk	Rewards relevant passages "near the top" — the brief's exact wording

Metric	Definition	Why
Precision@k	Relevant chunks in the top k / k	Report it, but explain it is capped low when a question has one gold passage

$$\mathrm{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\mathrm{rank}_q} \qquad \mathrm{Recall@}k = \frac{|\text{gold} \cap \text{top-}k|}{|\text{gold}|}$$

Ablation [R]: run the same metrics for BM25-only, dense-only, hybrid (RRF) and, if built, hybrid + rerank, broken down by category. Expect BM25 to win on exact error codes and dense to win on paraphrases — the table *proves* why you chose hybrid.

Answer quality and behaviour (all questions):

Metric	Definition
Status accuracy	Predicted `status` = expected; show a confusion matrix
Not-found recall	Unanswerable questions correctly refused
False-refusal rate	Answerable questions wrongly refused
Clarification rate	Ambiguous questions that got a clarifying question
Conflict recall	Conflict questions with `status = conflict` **and** both sides cited
Conflict false-positive rate	Non-conflict questions (incl. the scoped one) flagged as conflicts
Citation precision	Citations that point to a gold-relevant chunk
Answer keyword coverage	Expected keywords present in the answer
Hallucinations caught	Claims or citation ids dropped by the validator — evidence the guardrail works
Latency p50 / p95	Retrieval-only and end-to-end

Groundedness method: every displayed claim has a validated citation by construction; you measure (a) claims dropped by the validator, (b) a manual check of 10 answers by a member who didn't write the prompt, and **[O]** an LLM-as-judge pass on a sample. State that LLM-as-judge shares the generator's biases.

Report outline (`eval/reports/eval_report.md`): test set and how it was built → retrieval table + ablation → status confusion matrix → conflict and fallback results → 5+ worked examples with full outputs → failure cases (3–5, honest) → limitations (small set, synthetic corpus, thresholds tuned on the same set, no OCR).

9.4 Differentiators that are realistic in the time

Feature	Label	Cost	Why judges notice
Retrieval ablation table by category	[R]	~30 min (M3)	Proves design choices with numbers
"Why this answer" debug panel: dense/BM25/RRF ranks, gate decision, confidence reasons	[R]	~30 min (M3, data from `debug`)	Makes the system explainable live
Version-aware conflict cards with "newer" badge	Part of FR-06	~20 min (M3)	The hardest requirement, visibly done
Pre-LLM refusal with "LLM call avoided" note	[R]	free	Shows cost and hallucination awareness
Validator counter: "2 unsupported claims removed"	[R]	~15 min	Shows the guardrail working, not just claimed
Extractive degraded mode	[R]	~20 min (M1)	Demo survives an API outage
Cross-encoder rerank	[O]	~45 min	Measurable lift in the ablation
Document viewer with highlighted excerpt	[O]	~45 min	Citation clicks feel real

Not worth it in 5 hours: fine-tuning embeddings, knowledge graphs, multi-agent pipelines, OCR, authentication.

Stage 10 — Final implementation blueprint

The blueprint is the stages above in one place; this section adds what they don't cover — build sequence, integration and testing plan, demo script, risks, and the pitch.

10.1 Blueprint index

Item	Where
1. Final architecture	Stage 3 diagram and 3.1 component table
2. Final technology stack	4.7
3. Component responsibilities	5.3 matrix
4. Folder structure	Stage 7
5. API contracts	Stage 6
6. Data flow	3.2 sequence diagram
8. Schedule	Stage 8

10.2 Implementation sequence (dependency order)

1. `common/schemas.py` + fixture JSON + stub engine → unblocks everyone.
2. MD/TXT parsers + chunker → unblocks real retrieval.
3. Embeddings + index + hybrid retriever → unblocks retrieval eval.
4. API upload → ingestion → index; UI on real API → **Integration 1**.
5. LLM client + generator + validator → real cited answers.
6. Confidence gate → not-found and clarify paths.
7. Conflict rules + merge → conflict path.
8. PDF/HTML parsers + metadata → full format coverage and version data for conflicts.
9. Full eval → threshold tuning → **Integration 2** → freeze.
10. Report, README, demo.

10.3 Integration plan

- **Contract first:** `schemas.py` and `eval/fixtures/sample_query_result.json` are committed at 0:30. M1's stub returns the fixture; M3's mock mode reads the fixture; M2's test validates the fixture against `QueryResult`. If all three agree on the fixture, they agree on the contract.
- **Two scheduled integrations** (1:45 and 3:50), each with a written pass condition (Stage 8). Everyone stops coding for them.
- **Integration owner** (M2 from 3:00) runs `scripts/smoke_test.py` after every merge: reset → upload corpus → 5 questions → assert each `status`.
- **Backup path:** `frontend/api_client.py` has a `DIRECT` mode that imports `rag.engine` and `ingestion.pipeline` instead of calling HTTP.

Known integration risks, identified now:

Risk	Prevention
Schema drift (someone renames a field)	Frozen file, additive-only changes, fixture test
Import path errors (`ModuleNotFoundError`)	Run everything from repo root; empty `__init__.py` in every package
Embedding model loaded per request (slow)	Load once at FastAPI startup (lifespan event) and cache in `embeddings.py`
File upload silently empty	`python-multipart` installed; test with `curl -F` early
Streamlit reruns re-uploading files	Keep uploaded file names in `st.session_state`; upload only on button click
Index lost on restart	Persist after every `index_chunks`; load at startup
LLM returns invalid JSON	Structured-output mode + one repair attempt + extractive fallback
Page numbers off by one	PyMuPDF pages are 0-based; store `page + 1`

10.4 Testing strategy

Level	What	Owner	When
Unit	Parser per format; chunker never crosses top-level sections; retriever finds known chunk; validator drops fake ids; gate refuses nonsense	M2, M1	As each module lands
Contract	Fixture JSON validates against `QueryResult`	M2	0:30, then on every schema change
Integration	`smoke_test.py` end to end over HTTP	M2	After every merge from 3:00
System	Full eval run (retrieval + behaviour metrics)	M3	3:00 (retrieval), 3:50 (full)
Manual	Demo checklist on a fresh `/reset`	All	4:35

10.5 Demonstration workflow (6–7 minutes)

1. **Frame the problem (20 s):** "Answers are scattered and sometimes contradict each other. We built an assistant that proves every claim and admits what it doesn't know."
2. **Upload (40 s):** drag in the corpus — PDF, HTML, MD, TXT. Point at detected title, version and effective date per document, and chunk counts.
3. **Direct question (40 s):** "What does ERR-429 mean?" → cited answer; click the citation → section and excerpt.
4. **Paraphrase (40 s):** "My teammate can't get into their account — what should an admin do?" → password-reset passage; open the debug panel: dense found it, BM25 didn't, hybrid kept it.
5. **Conflict (60 s):** "What's the refund window for annual plans?" → conflict card: 30 days (Policy v2.0, 2025) vs 14 days (FAQ, 2023), newer badge, no silent choice.
6. **Ambiguous (30 s):** "What's the limit?" → clarifying question with options.
7. **Unanswerable (30 s):** "What's the parental leave policy?" → clear not-found, "LLM call avoided".
8. **Evaluation (90 s):** Recall@5, MRR, ablation table, status confusion matrix, one honest failure case.
9. **Invite the judge** to ask their own question. Close on the architecture diagram.

Pre-warm every scripted question so answers come from the LLM cache instantly.

10.6 Technical risks and backup plans

Risk	Likelihood	Impact	Backup
LLM free-tier quota or rate limit hit	High	Answers fail	Disk cache; second provider; extractive degraded mode
Venue Wi-Fi poor	Medium	No installs, no LLM	Install and download model at 0:00; phone hotspot; cached demo answers; backup video
PyTorch install slow	Medium	Lost 20 min	Use `fastembed` (ONNX) for the same bge model
PDF heading detection poor	Medium	Weak section citations	Fall back to page-level sections; page numbers still cite
Thresholds misjudge answerable vs not	Medium	Wrong refusals	Tune on eval; show score; conservative default
Conflict rule false positives	Medium	Spurious warnings	Similarity filter + LLM confirmation + "possible conflict" label
API breaks late	Low	UI dead	Streamlit `DIRECT` mode
Running out of time	High	Missing deliverables	Cut list (8.3); freeze at 4:10; docs written in parallel from 4:10
Sensitive data on free tier	Low (synthetic corpus)	Policy issue	Use only synthetic/public documents; note in README

10.7 Pitch for the judges (about 75 seconds)

> Employees waste time hunting across manuals, FAQs and release notes — and when those documents disagree, they can't tell which answer is current. Our assistant ingests PDF, HTML, Markdown and text files, keeps each document's structure, version and date, and splits them into section-aware passages.
>
> When you ask a question, we search two ways at once — keyword search for exact terms like error codes, and semantic search for paraphrases — and fuse the results. Before we ever call the language model, a confidence gate checks whether the evidence is strong enough. If it isn't, we say so, or ask a clarifying question, instead of guessing.
>
> When we do answer, the model may only use numbered evidence, and must cite it on every sentence. We then validate every citation and drop anything unsupported. If two sources disagree — say, an old FAQ and a new policy — we show both sides with their versions and dates rather than silently picking one.
>
> Finally, we measured it: a labelled set of 30 questions, including conflicting, ambiguous and unanswerable ones, with Recall@5, MRR, and an ablation showing why hybrid retrieval beats either method alone.

Handoff plan for Claude Code

Copy the shared block into Claude Code on every laptop first, then each member pastes their own task block. The shared block is the source of truth; if the code assistant proposes something that contradicts it, the block wins.

Shared block (all members)

```
# PROJECT HANDOFF — Enterprise Documentation Q&A Assistant (HCLTech
hackathon)

## Goal
Web app + API: upload PDF/HTML/MD/TXT documents, ask natural-language
questions, get concise
answers grounded ONLY in retrieved passages, with per-claim citations (doc,
section, page,
excerpt, link), visible conflicts between sources, confidence + not-
found/clarify fallback,
```

```
and an evaluation report. Team of 3, ~5 hours. Prototype, not production.

## Hard rules for the coding assistant
- Python 3.11. Pydantic v2. Type hints. Small functions. No file over ~300
lines.
- NO LangChain, NO LlamaIndex, NO vector database server. NumPy + rank_bm25
only.
- common/schemas.py is FROZEN. Only additive optional fields, and only if I
say so.
- Only edit files in my owned folders (listed in my member block).
- Every module must be runnable/testable on its own (pytest test or __main__
demo).
- No secrets in code. Read keys from .env via python-dotenv.
- Never let an LLM failure raise to the API: return a degraded result
instead.
- Do not add features not listed here without asking.

## Requirement -> module traceability
FR-01 ingestion/parsers.py, metadata.py | FR-02 ingestion/cleaner.py,
chunker.py
FR-03 rag/retriever.py (BM25 + dense + RRF) | FR-04 rag/generator.py,
prompts.py
FR-05 rag/validator.py | FR-06 rag/conflicts.py | FR-07 rag/confidence.py
FR-08 eval/run_eval.py, metrics.py

## Stack
Streamlit UI -> FastAPI (uvicorn, python-multipart) -> in-process engine.
PyMuPDF (PDF), BeautifulSoup4+lxml (HTML), own parsers (MD/TXT).
sentence-transformers with BAAI/bge-small-en-v1.5 (query prefix:
"Represent this sentence for searching relevant passages: "). rank_bm25.
NumPy.
LLM: Gemini Flash (google-genai) primary, Groq fallback; provider + model
from .env.
SQLite (stdlib) registry; files in data/uploads; index in data/index; LLM
cache in data/llm_cache.

## Folders and owners
common/schemas.py (ALL, M1 gatekeeper) | config.py (M1)
ingestion/ backend/ tests/test_ingestion.py tests/test_api.py
scripts/smoke_test.py (M2)
rag/ tests/test_retrieval.py scripts/check_llm.py (M1)
frontend/ eval/ README.md docs/ (M3)
Run from repo root: uvicorn backend.main:app --reload --port 8000 ; streamlit
```

```
run frontend/app.py

## Schemas (common/schemas.py)
UserMeta: version?, effective_date? (YYYY-MM-DD), product?, doc_kind?
DocumentRecord: doc_id (sha256[:12] of bytes), filename, title, doc_type
(pdf|html|md|txt),
  doc_kind (manual|faq|release_notes|policy|support_article|other), version?,
effective_date?,
  product?, pages?, num_chunks, uploaded_at, source_url
Block (ingestion internal): text, section_path: list[str], page?: int (1-
based)
Chunk: chunk_id ("{doc_id}:{ordinal:04d}"), doc_id, doc_title, doc_type,
doc_kind,
  section_path, page_start?, page_end?, ordinal, text, embed_text, version?,
effective_date?, word_count
Filters: doc_ids?, version?, product?
ScoredChunk: chunk, dense_score, dense_rank?, bm25_rank?, rrf_score,
rerank_score?, final_score
Citation: id (1..n), chunk_id, doc_id, doc_title, filename, section, page?,
version?,
  effective_date?, excerpt (<=300 chars), source_url, relevance
Claim: text, citation_ids: list[int], support (supported|weak)
ConflictPosition: citation_id, statement, version?, effective_date?
Conflict: type (version_sensitive|value_mismatch|contradiction), topic,
summary,
  positions: list[ConflictPosition], resolution_hint?, detected_by
(rules|llm|rules+llm)
Confidence: level (high|medium|low), score (0-1), reasons: list[str]
QueryRequest: question (1-1000 chars), top_k=5, filters?, debug=false
QueryResult: query_id, question, status
(answered|partial|conflict|needs_clarification|not_found),
  answer (text with [n] markers), claims, citations, conflicts, confidence,
  clarifying_question?, degraded: bool, model?, timings_ms: dict, debug?:
dict
IndexStats: documents, chunks
ErrorBody: code, message, details: dict  -> HTTP body {"error": ErrorBody}

## Engine API (rag/engine.py) — the ONLY rag import allowed outside rag/
index_chunks(doc: DocumentRecord, chunks: list[Chunk]) -> IndexStats
delete_document(doc_id: str) -> IndexStats
retrieve(question: str, k: int = 5, filters: Filters | None = None) ->
list[ScoredChunk]
query_documents(question: str, top_k: int = 5, filters=None, debug=False) ->
```

```
QueryResult
index_stats() -> IndexStats ; reset() -> IndexStats

## Ingestion API (ingestion/pipeline.py)
process_file(path: Path, user_meta: UserMeta | None = None) ->
tuple[DocumentRecord, list[Chunk]]
raises IngestionError(code, message); codes: UNSUPPORTED_FORMAT,
EMPTY_DOCUMENT, PARSE_FAILED

## HTTP API (backend/main.py)
GET /health | POST /upload (multipart files[] + optional form meta) -> per-
file indexed|duplicate|failed
GET /documents | GET /documents/{id}/file | DELETE /documents/{id} | POST
/query | POST /reset
not_found and degraded answers are HTTP 200. Errors: 413 FILE_TOO_LARGE, 415
UNSUPPORTED_FORMAT,
422 EMPTY_DOCUMENT/PARSE_FAILED/INVALID_REQUEST, 404 DOCUMENT_NOT_FOUND, 409
INDEX_EMPTY, 500 INTERNAL_ERROR

## config.py defaults (thresholds are PLACEHOLDERS — tune on eval)
EMBED_MODEL="BAAI/bge-small-en-v1.5"; DENSE_K=20; BM25_K=20; RRF_K=60;
TOP_K=5
CHUNK_TARGET_WORDS=300; CHUNK_MAX_WORDS=450; CHUNK_OVERLAP_WORDS=40;
MIN_SECTION_WORDS=40
T_LOW (not-found gate on top dense cosine)=0.45; CONF_HIGH=0.70;
CONF_MEDIUM=0.45
CONFLICT_SIM=0.75; MAX_FILE_MB=20; USE_RERANKER=False
Note: bge cosine scores are compressed (unrelated text often scores 0.4-0.6).
Choose T_LOW
from eval: midpoint between max score of unanswerable questions and min of
answerable ones.

## Milestones (definition of done)
A 0:30  schemas.py + eval/fixtures/sample_query_result.json committed; stub
engine returns fixture; LLM keys verified
B 1:45  retrieve() returns sensible top-5 on corpus; API /upload indexes
MD/TXT
I1 2:05 UI uploads corpus via API and shows real passages (extractive mode)
C 3:00  cited LLM answers end to end; Recall@5 and MRR computed
I2 4:10 gate, conflicts, all 4 formats, full eval report — FEATURE FREEZE
D 5:00  README, eval report, 5+ evidence-backed examples, demo rehearsed,
backup video
```

```
## Cut order if late
reranker -> claim-support check -> LLM conflict confirmation -> rule-based
clarify ->
HTML tables/TXT heading heuristics -> delete/reset/filters -> UI polish.
NEVER cut: claim citations, not-found gate, version-conflict rule,
Recall@k/MRR eval, README.
```

Member 1 block — AI/RAG (you)

```
I am Member 1. I own rag/, config.py, tests/test_retrieval.py,
scripts/check_llm.py.
Build in this order, one module at a time, each with a quick test:
1. rag/engine.py STUB: query_documents returns QueryResult parsed from
   eval/fixtures/sample_query_result.json; index_chunks counts chunks. Commit
immediately.
2. scripts/check_llm.py: one tiny JSON-mode call per provider; print model +
latency.
3. rag/embeddings.py: load model once (module-level cache); encode(texts,
is_query) ->
   L2-normalised float32 array; query prefix for bge.
4. rag/index.py: chunks list + NumPy matrix + BM25Okapi (tokenise lowercase,
keep [a-z0-9-]);
   add, delete by doc_id, save/load data/index/{chunks.jsonl,
embeddings.npy}.
5. rag/retriever.py: dense top DENSE_K, BM25 top BM25_K, RRF (k=60), filters,
keep ranks/scores,
   optional cross-encoder rerank behind USE_RERANKER, return top_k
ScoredChunk.
6. rag/llm.py: complete_json(system, user) -> dict. Provider from env,
structured JSON output,
   timeout 30s, 2 retries with backoff, fallback provider, disk cache by
sha256(prompt+model).
7. rag/prompts.py + rag/generator.py: evidence blocks "[S1] doc | section |
page | version | date\n text".
   System rules: use ONLY evidence; every claim cites >=1 S id; if sources
disagree describe
   each side with its source and do not choose; insufficient evidence ->
status not_found;
   underspecified question -> needs_clarification + one question. Output
JSON:
   {status, claims:[{text, evidence:["S1"]}], conflicts:[{type, topic,
summary, evidence:["S2","S4"]}],
```

```
    clarifying_question, missing_info}. One repair attempt on bad JSON.
8. rag/validator.py: drop unknown S ids and uncited claims; excerpt = most
similar sentence(s)
   in cited chunk (<=300 chars); renumber [1..n] by first use; build Citation
objects;
   source_url "/documents/{doc_id}/file" + "#page=N" for PDFs; count dropped
claims.
9. rag/confidence.py: score = 0.6*norm(top dense) + 0.2*agreement(top chunk
in both top-5)
   + 0.2*support(hits near top score, cap 3); levels from config; gate
decisions:
   not_found (below T_LOW, no LLM call), clarify (no version/product cue in
question AND top
   hits split across products/features with close scores), else answer.
Return reasons.
10. rag/conflicts.py: version rule (same doc family/kind, different version
or date among
   relevant hits); value rule (different docs, chunk cosine >= CONFLICT_SIM,
same unit
   different number: days, requests/min, GB, users, $). Pass as hints to
prompt; merge with
   LLM conflicts; keep rule value-mismatches the LLM ignored as "possible
conflict".
11. rag/engine.py real implementation; extractive fallback when LLM fails
(degraded=true).
12. Tune thresholds with eval output; then optional reranker and claim-
support check.
```

Member 2 block — Backend and ingestion

```
I am Member 2. I own backend/, ingestion/, tests/test_ingestion.py,
tests/test_api.py,
tests/fixtures/, scripts/smoke_test.py, requirements.txt, .gitignore.
Build in this order:
1. FastAPI skeleton (backend/main.py) with all endpoints wired to the STUB
rag.engine;
   load engine/index once at startup (lifespan). Swagger at /docs must work.
2. backend/storage.py: save upload to data/uploads/{doc_id}_{filename};
SQLite table documents
   (all DocumentRecord fields); duplicate detection by doc_id.
3. backend/errors.py: error codes -> HTTP status; handlers returning
{"error": {...}}.
```

```
4. ingestion/parsers.py: parse_md (ATX headings, YAML front matter -> meta,
keep code fences),
   parse_txt (CAPS lines, "1.2 Title", underline headings; else blank-line
paragraphs)
   FIRST and hand to M1; then parse_html (drop
script/style/nav/header/footer; walk h1-h6,
   p, li, pre, table rows in order); then parse_pdf (PyMuPDF dict spans;
heading = font size
   >= 1.15x body median or bold short line; page 1-based; strip lines
repeated on >50% pages).
   All return list[Block]. No text -> EMPTY_DOCUMENT.
5. ingestion/cleaner.py: NFKC, whitespace, de-hyphenate line breaks.
6. ingestion/metadata.py: title (front matter / <title> / first heading /
filename);
   version via regex on filename + first 2000 chars (v2, Version 3.1);
effective_date near
   effective|updated|revised|release; doc_kind from filename keywords;
UserMeta overrides.
7. ingestion/chunker.py: never cross top-level section; target 300 words, max
450; split on
   paragraph then sentence with 40-word overlap; merge sections < 40 words
into next sibling;
   embed_text = "{title} > {section path}\n{text}"; copy version/date onto
every chunk.
8. ingestion/pipeline.py process_file; wire /upload -> process_file ->
engine.index_chunks.
9. Tests: one tiny fixture per format; chunker boundary test; API tests via
TestClient;
   fixture JSON validates as QueryResult.
10. scripts/smoke_test.py: reset, upload eval/corpus, ask 5 questions, assert
statuses.
```

Member 3 block — Frontend, evaluation, demo

```
I am Member 3. I own frontend/, eval/, README.md, docs/.
Build in this order:
1. eval/corpus/: 8-10 short documents about a FICTIONAL product "Nimbus
Workspace" in all four
   formats with planted cases: refund 30 days (policy v2.0, 2025) vs 14 days
(FAQ, 2023);
   API rate limit 100/min (api ref v2.4, 2023) vs 300/min (release notes
v3.0, 2025);
```

```
   retention 90 days (security FAQ) vs 180 days Enterprise-only (scoped, NOT
a conflict);
   error codes ERR-401/403/429; plan limits table; synonyms (SSO/SAML,
seats/users,
   workspace/tenant); gaps: parental leave, on-prem pricing, offline mode,
DPA signing.
2. eval/fixtures/sample_query_result.json (conflict example from the shared
contract).
3. frontend/api_client.py with modes MOCK (fixture), HTTP (default), DIRECT
(import engine).
4. frontend/app.py (Streamlit): sidebar upload (multi-file, optional
version/date) + document
   list with version/date/chunks; chat with st.chat_message; status badge;
confidence badge
   + reasons; answer with [n] markers; citation cards (doc, section, page,
version, excerpt,
   link); conflict panel showing each position side by side with "newer"
badge;
   clarifying question and not-found styling; "Why this answer" expander from
debug.
5. eval/questions.jsonl: ~30 questions: 8 direct, 6 paraphrase, 4
terminology, 4 conflict,
   4 ambiguous, 4 unanswerable, +1 scoped non-conflict. Gold = {doc, evidence
substring}.
   Relevant chunk = same doc AND text contains evidence (case-insensitive).
6. eval/metrics.py + eval/run_eval.py: retrieval Recall@1/3/5, Hit@k, MRR,
Precision@k by
   category, ablation (bm25-only, dense-only, hybrid[, rerank]) via
retrieve() modes;
   end-to-end status accuracy + confusion matrix, not-found recall, false-
refusal rate,
   clarification rate, conflict recall + false positives, citation precision,
keyword
   coverage, validator drops, latency p50/p95. Write
eval/reports/eval_report.md + metrics.json.
   Sleep between LLM calls to respect rate limits; rely on M1's LLM cache for
reruns.
7. README: setup, run, supported formats, ingestion/chunking, retrieval,
generation,
   conflicts, confidence, evaluation summary, assumptions (synthetic data),
trade-offs, limits.
8. docs/demo_script.md, screenshots, backup screen recording.
```